

PROJET NoSQL — GROUPE 11

Site web d'Annonces Immobilières

Rapport final

Membres du groupe

MBA MENGOUA Evann Terrence

AYONGA TOUGANDE Noëlle-Wilda

BIAKBA Yvette

Sommaire

1. Introduction
2. Contexte métier
3. Modélisation NoSQL
4. Implémentation
5. Requêtes et agrégations
6. Indexation
7. Analyse des performances
8. Conclusion

Introduction

Le présent rapport documente la conception et la mise en œuvre d'un site web d'annonces immobilières s'appuyant sur une base de données NoSQL. La plateforme couvre Libreville et ses quartiers (Akanda, Glass, Nzeng-Ayong) et permet aux utilisateurs de publier des biens en location ou en vente, accompagnés de photos, d'un prix et d'une localisation, puis de les rechercher par mots-clés ou par proximité géographique.

Le choix d'une base documentaire MongoDB répond à deux besoins majeurs du domaine immobilier : la recherche plein-texte sur des descriptions de biens et le traitement géospatial des localisations. Ces deux fonctionnalités exploitent des mécanismes d'indexation spécialisés que les bases relationnelles traditionnelles gèrent moins naturellement.

Le rapport est organisé en huit parties. Après ce premier chapitre introductif, le **contexte métier** (chapitre 2) précise les besoins fonctionnels. La **modélisation NoSQL** (chapitre 3) détaille les collections et les choix d'embedding et de référencement. L'**implémentation** (chapitre 4), les **requêtes et agrégations** (chapitre 5) et l'**indexation** (chapitre 6) présentent la réalisation technique. Enfin, l'**analyse des performances** (chapitre 7) mesure l'impact concret de l'indexation, avant la **conclusion** (chapitre 8).

2. Contexte métier

Le site web s'adresse à un marché local : des utilisateurs publient des biens et peuvent rechercher un logement à louer ou à acheter dans une zone géographique précise. La pertinence d'une plateforme immobilière repose largement sur la qualité de la recherche : un utilisateur veut trouver rapidement les biens correspondant à ses critères et, surtout, situés près de l'endroit qui l'intéresse.

2.1 Besoins fonctionnels

- Publication de biens en location ou en vente, avec titre, description, prix, surface, nombre de pièces, photos et statut de disponibilité.
- Recherche plein-texte par mots-clés portant sur le titre et la description des annonces.
- Recherche de proximité : trouver les biens situés dans un rayon donné autour d'un point (par exemple la position de l'utilisateur ou un point d'intérêt).
- Filtres combinés : type de transaction (location ou vente), fourchette de prix et nombre de pièces.
- Statistiques par secteur, notamment le prix moyen au mètre carré par quartier.

2.2 Enjeux techniques

Deux familles de requêtes structurent le système. D'une part, la **recherche textuelle**, qui exige un index de type texte pour rester rapide à mesure que le catalogue grandit. D'autre part, les **requêtes géospatiales**, qui nécessitent un index 2dsphere pour calculer efficacement les distances sur la surface terrestre. À cela s'ajoutent les filtres multi-critères, fréquents dans l'interface, dont la performance dépend directement d'un index composé adapté.

3. Modélisation NoSQL

La base de données est articulée autour de trois entités principales. La modélisation cherche un équilibre entre des documents légers, faciles à maintenir, et des accès rapides limitant le nombre de jointures.

3.1 Description des collections

- **Collection annonces.** Collection centrale de la plateforme, elle stocke l'ensemble des informations descriptives et techniques relatives aux biens immobiliers disponibles : titre, description, prix, surface, nombre de pièces, photos et statut de disponibilité. Elle inclut également les données de localisation au format GeoJSON pour permettre le traitement géospatial.
- **Collection quartiers.** Cette collection agit comme un référentiel géographique : elle regroupe les zones d'intervention de la plateforme. Elle permet de standardiser les zones et sert de base aux calculs de statistiques et aux agrégations par secteur.
- **Collection utilisateurs.** Cette collection gère les données d'identité des membres. Elle centralise les informations relatives aux annonceurs et aux clients (nom, email, numéro de téléphone et paramètres de sécurité), distingue les comptes et permet de gérer les droits d'accès.

3.2 Schémas de données

Pour garantir l'intégrité et la cohérence des données, la structure de chaque collection a été définie comme suit.

Schéma : annonces

```
// Collection annonces
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "quartierId": "ObjectId",
  "titre": "String",
  "description": "String",
  "type": "String",
  "prix": "Int",
  "surface": "Int",
  "nbr_pieces": "Int",
  "localisation": "GeoJSON",
  "photos": ["String"],
  "disponible": "Boolean",
  "createdAt": "Date"
}
```

Schéma : utilisateurs

```
// Collection utilisateurs
{
  "_id": "ObjectId",
  "nom": "String",
  "email": "String",
  "telephone": "String",
  "password": "String",
  "photo_profil": "String",
  "etat": "Int",
  "createdAt": "Date"
}
```

Schéma : quartiers

```
// Collection quartiers
{
  "_id": "ObjectId",
  "nom": "String",
  "ville": "String",
  "description": "String"
}
```

3.3 Relations entre les collections

- **Relation annonces → utilisateurs.** Relation de type « Many-to-One » : plusieurs annonces peuvent appartenir à un seul utilisateur. Le champ `userId` de la collection `annonces` contient l'identifiant (`_id`) de l'utilisateur. Cette structure associe chaque bien à son propriétaire légitime tout en isolant les données personnelles (email, mot de passe) dans la collection dédiée.
- **Relation annonces → quartiers.** Relation également de type « Many-to-One » : plusieurs annonces peuvent être situées dans un même quartier. Le champ `quartierId` pointe vers l'identifiant du quartier correspondant. Ce lien standardise les données géographiques, facilite les recherches par secteur et permet de générer des statistiques précises (prix moyen, nombre de biens disponibles) sans risque d'erreur de saisie sur les noms de zones.

3.4 Choix techniques : embedding et référencement

Une stratégie hybride a été adoptée, adaptant la structure de la donnée à son usage quotidien dans la plateforme.

- **Référencement (privilégié).** Utilisé pour les relations entre annonces, utilisateurs et quartiers : seul l'identifiant est stocké dans le document annonce. Cette approche est avantageuse pour la maintenance : si un utilisateur modifie ses coordonnées ou si un quartier est renommé, la mise à jour s'effectue à un seul endroit, sans parcourir des milliers d'annonces pour corriger des doublons. Elle conserve aussi des documents annonces légers et optimisés, rendant la base plus rapide et plus simple à administrer sur le long terme.
- **Embedding (pour la localisation).** Les coordonnées géospatiales sont intégrées directement dans le document annonce, au format GeoJSON. La localisation étant indissociable du bien lors de la consultation, cette structure permet de récupérer toutes les informations utiles en une seule requête atomique. Elle simplifie considérablement les requêtes de proximité, car MongoDB n'a pas besoin d'effectuer de jointure coûteuse entre collections, garantissant la fluidité de l'affichage sur une carte.

4. Implémentation

L'application repose sur une base MongoDB et une couche applicative qui expose des points d'accès (endpoints) pour la publication, la recherche et la consultation des annonces. Les documents sont créés selon les schémas décrits au chapitre 3.

4.1 Création d'une annonce

À la création, l'annonce stocke sa localisation sous forme de point GeoJSON, structure imposée par l'index géospatial. L'ordre des coordonnées est toujours [longitude, latitude].

```
db.annonces.insertOne({
  titre: "Villa avec piscine à Akanda",
  description: "Belle villa meublée, 4 chambres, jardin",
  type: "location",
  prix: 350000,
  surface: 180,
  pieces: 5,
  statut: "disponible",
  location: { type: "Point", coordinates: [9.5012, 0.5421] },
  userId: ObjectId("..."),
  quartierId: ObjectId("...")
})
```

4.2 Endpoints principaux

- POST /annonces — création d'une annonce (avec localisation GeoJSON).
- GET /annonces/search?q=... — recherche plein-texte par mots-clés.
- GET /annonces/near?lng=...&lat=...&rayon=... — recherche de proximité.
- GET /annonces?type=...&prixMin=...&prixMax=...&pieces=... — filtres combinés.
- GET /quartiers/stats — prix moyen au m² par quartier.

5. Requêtes et agrégations

5.1 Recherche plein-texte

La recherche par mots-clés s'appuie sur l'index texte défini sur le titre et la description. Le résultat est trié par score de pertinence, calculé par MongoDB.

```
db.annonces.find(
  { $text: { $search: "villa piscine" } },
  { score: { $meta: "textScore" } }
).sort(
  { score: { $meta: "textScore" } }
)
```

5.2 Recherche de proximité

La requête de proximité retourne les biens situés dans un rayon de N kilomètres autour d'un point. L'opérateur \$near, qui s'appuie sur l'index 2dsphere, renvoie les annonces déjà triées de la plus proche à la plus éloignée.

```
db.annonces.find({
  location: {
    $near: {
      $geometry: { type: "Point", coordinates: [lng, lat] },
      $maxDistance: 5000 // 5 km, en mètres
    }
  }
})
```

5.3 Filtres combinés

Plusieurs critères sont croisés dans une même requête pour affiner les résultats selon les besoins de l'utilisateur : type de transaction, fourchette de prix et nombre de pièces, éventuellement combinés à la recherche plein-texte.

```
db.annonces.find({
  type: "location",
  prix: { $gte: 150000, $lte: 400000 },
  pieces: { $gte: 3 }
})
```

5.4 Agrégation : prix moyen au m² par quartier

Cette agrégation regroupe les annonces par quartier et calcule, pour chacun, le prix moyen au mètre carré. Elle constitue un indicateur clé pour situer un bien par rapport au marché local.

```
db.annonces.aggregate([
  { $group: {
    _id: "$quartierId",
    prixM2Moyen: { $avg: { $divide: ["$prix", "$surface"] } },
    nbBiens: { $sum: 1 }
  } },
  { $sort: { prixM2Moyen: -1 } }
])
```

6. Indexation

L'indexation est au cœur des performances de la plateforme. Trois index spécialisés répondent aux trois familles de requêtes identifiées.

6.1 Index texte

Un index de type texte est créé sur le titre et la description. Une pondération plus forte est attribuée au titre, jugé plus déterminant que la description pour la pertinence.

```
db.annonces.createIndex(
  { titre: "text", description: "text" },
  { weights: { titre: 5, description: 1 } }
)
```

6.2 Index géospatial 2dsphere

L'index 2dsphere porte sur le champ location et permet les requêtes de proximité avec des distances calculées sur une sphère, donc réalistes à l'échelle géographique.

```
db.annonces.createIndex({ location: "2dsphere" })
```

6.3 Index composé pour les filtres

Le filtre combiné le plus fréquent de l'interface porte sur le type et le prix. Un index composé sur ces deux champs accélère fortement cette recherche multi-critères, comme le démontre le chapitre suivant.

```
db.annonces.createIndex({ type: 1, prix: 1 })
```

7. Analyse des performances

Afin de démontrer l'efficacité de la stratégie d'indexation, une analyse a été menée sur la collection annonces, autour de l'un des filtres les plus fréquents de l'interface : le filtrage combiné par type et par prix. La méthode compare l'exécution de la même requête avant et après la création de l'index composé, en s'appuyant sur la commande explain() de MongoDB.

7.1 Avant indexation : COLLSCAN

Sans index sur les critères de recherche, le moteur effectue un balayage complet de la collection pour filtrer les documents pertinents. L'exécution révèle les indicateurs suivants : stage COLLSCAN, 52 documents examinés (totalDocsExamined), 26 résultats retournés (nReturned).

Le moteur a donc examiné l'intégralité des 52 documents stockés pour n'en retenir que 26. Ce comportement, propre au COLLSCAN, traduit une complexité linéaire $O(n)$ qui induit une charge inutile sur les ressources système (I/O et CPU) et limite la scalabilité de l'application à mesure que le volume de données augmente.

```
>_MONGOSH
> use immoLib
< switched to db immoLib
> db["annonces"].find({ type: "location", prix: { $lt: 500000 } }).explain('executionStats')
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'immoLib.annonces',
    parsedQuery: {
      '$and': [
        {
          type: {
            '$eq': 'location'
          }
        },
        {
          prix: {
            '$lt': 500000
          }
        }
      ]
    },
    indexFilterSet: false,
    queryHash: 'C05C7C69',
    planCacheShapeHash: 'C05C7C69',
    planCacheKey: 'CB6D4E2D',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 26,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 52,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: {
      '$and': [
        {
          type: {
            '$eq': 'location'
          }
        },
        {
          prix: {
            '$lt': 500000
          }
        }
      ]
    },
    nReturned: 26,
    executionTimeMillisEstimate: 0,
    works: 53,
    advanced: 26,
    needTime: 26,
```

7.2 Après indexation : IXSCAN

Après création d'un index composé sur les champs type et prix, les données sont structurées sous forme d'arbre, ce qui permet au moteur de localiser les documents pertinents sans parcourir toute la

collection. La même requête, réexécutée, présente les indicateurs suivants : stage IXSCAN, 26 documents examinés, 26 résultats retournés.

Le nombre de documents examinés est désormais strictement égal au nombre de résultats retournés : l'efficacité est optimale, car le moteur ne traite plus aucune donnée inutile. L'index pointe directement vers les documents ciblés via une structure B-Tree, réduisant fortement la charge CPU et I/O.

```
>_MONGOSH
{
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 26,
    "executionTimeMillis": 0,
    "totalKeysExamined": 26,
    "totalDocsExamined": 26,
    "executionStages": {
      "isCached": false,
      "stage": "FETCH",
      "nReturned": 26,
      "executionTimeMillisEstimate": 0,
      "works": 27,
      "advanced": 26,
      "needTime": 0,
      "needYield": 0,
      "saveState": 0,
      "restoreState": 0,
      "isEOF": 1,
      "docsExamined": 26,
      "alreadyHasObj": 0,
      "inputStage": {
        "stage": "IXSCAN",
        "nReturned": 26,
        "executionTimeMillisEstimate": 0,
        "works": 27,
        "advanced": 26,
        "needTime": 0,
        "needYield": 0,
```

Cette comparaison prouve que l'indexation est essentielle à l'optimisation de la plateforme. Le passage d'un balayage complet (COLLSCAN) à une recherche indexée (IXSCAN) divise par deux le travail du serveur (de 52 à 26 documents analysés). À cette échelle, le gain reste modéré, mais le COLLSCAN ayant un coût qui croît linéairement avec le volume, l'écart se creuse à mesure que le catalogue grandit : c'est précisément ce qui garantit la rapidité et la scalabilité du projet, même avec un volume important d'annonces.

Conclusion

Ce projet a permis de concevoir une plateforme d'annonces immobilières adaptée au contexte de Libreville, en exploitant les atouts d'une base de données documentaire pour répondre à des besoins de recherche textuelle et géospatiale.

La modélisation a reposé sur une stratégie hybride assumée : **référencer** les utilisateurs et les quartiers pour garder des documents légers et faciles à maintenir, et **embarquer** la localisation pour traiter les requêtes de proximité en un seul accès. Les requêtes et agrégations mises en place — recherche par mots-clés, recherche \$near, filtres combinés et prix moyen au m² par quartier — couvrent l'ensemble des fonctionnalités attendues.

Enfin, l'analyse des performances a démontré, mesures à l'appui, l'impact concret de l'indexation : le passage du COLLSCAN à l'IXSCAN supprime le traitement des documents inutiles et garantit la montée en charge de l'application. Au-delà des résultats chiffrés sur un échantillon réduit, c'est la démarche qui importe : choisir le bon index pour chaque type de requête est la clé d'une plateforme rapide et durable.